YOLO'S TRAINING AND EVALUATION

File: Yolo_TrainingAndEvaluation_Jorge_Peguero_ITAI1378.ipynb

Jorge Peguero -W214777728

ITAI 1378

This notebook contains the process of training and evaluating a custom YOLO model from Ultralytics. It uses the YOLOv8n-cls for its efficiency and fast response and is prepared to access your Google Drive to save its results automatically.

Environment Setup:

```
%pip install ultralytics opencv-python requests

#Installing project dependencies
```

```
from google.colab import drive   #Accessing Google Drive to save our progress
```

```
from ultralytics import YOLO
import cv2
import requests
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
#Importing dependencies and required tools
```

```
import os
```

This notebook begins by installing and importing the required and necessary dependencies and tools:

- Ultralytics: Provides the YOLOv8 model and the necessary tools for custom training and evaluation.
- Drive: To connect our Google Colab project to Google Drive, so the training results are safely stored.
- Os: To download Kaggle's datasets.

Note: Some of the dependencies listed in this notebook are not used, but I decided to set up the same environment in both notebooks.

Connecting with Google Drive:

To effectively store the results from the training the notebook connects with Google Colab through this cell:

```
from google.colab import drive   #Accessing Google Drive to save our progress

drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Downloading the dataset:

```
import os

os.environ['KAGGLE_API_TOKEN'] = 'Insert Kaggle API Token'

!mkdir -p ~/.kaggle
!echo {os.environ['KAGGLE_API_TOKEN']} > ~/.kaggle/access_token
!chmod 600 ~/.kaggle/access_token

!kaggle datasets list -s "fire"    #Accesing Kaggle with API token and verifying API connection by listing some datasets
```

Using the OS library from Python, this code block sets up automated authentication between Google Colab and the Kaggle platform. It stores your credentials in the environment and creates a hidden directory with the required privacy permissions for the API to function correctly.

```
!kaggle datasets download -d elmadafri/the-wildfire-dataset
#downloading the required dataset from Kaggle
```

```
Dataset URL: https://www.kaggle.com/datasets/elmadafri/the-wildfire-dataset
License(s): Attribution 4.0 International (CC BY 4.0)
Downloading the-wildfire-dataset.zip to /content
100% 9.94G/9.94G [02:00<00:00, 88.5MB/s]
```

```
!unzip -q the-wildfire-dataset.zip
#Unzziping the dataset
```

Then, the notebook downloads The WildFire Dataset made by Ismail El Madafri. And uses the command “unzip” to extract the images and labels from the compressed files. This process automatically organizes the data into the structure required for training, validation, and testing.

Defining the model:

```
model = YOLO('yolov8n-cls.pt')
#Defining the model
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n-cls.pt to 'yolov8n-cls.pt': 100% ━━━━━━━━━━━━ 5.3MB 289.9MB/s 0.0s
```

```
print(len(list(model.model.parameters()))) #Verifying the model downloaded correctly and figuring out the number of parameters.
```

```
80
```

The notebook defines the model YOLOv8n for classification tasks and verifies that it is downloaded correctly and what is the number of parameters of the model. This is the base model that is going to be used for custom training.

Custom Training:

```
model.train(data ='/content/the_wildfire_dataset_2n_version',epochs = 20, patience = 5, freeze = 5, cls_pw = 0.25, save_period = 1, project ='/content/drive/MyDrive/CV_Project', name='CV_Project' )
```

Parameter	Value
Data	/content/the_wildfire _dataset _2n _version
Epochs	20
Patience	5
Freeze	5
Cls_pw	0.25
Save_period	1
Project	/content/drive/MyDrive/CV_Project
Name	CV_Project

Epochs: The wildfire Dataset is about 2700 images; 20 epochs is enough for the model to converge.

Patience: Stops training early if validation metrics do not improve for 5 consecutive epochs, preventing overfitting.

Freeze: Freezes the first 5 layers of the backbone. YOLOv8 is pretrained on ImageNet, early layers already detect generic features like shapes, textures…

Cls_pw: The dataset is unbalanced; this parameter reduces the penalty weight for the positive class.

Save_period: Saves a checkpoint every epoch, stored in Google Drive.

Calling the best model and running final evaluation:

```
best_model = YOLO('/content/drive/MyDrive/CV_Project/CV_Project/weights/best.pt')
```

```
evaluation = best_model.val(data ='/content/the_wildfire_dataset_2n_version', split = 'test', verbose =True, project ='/content/drive/MyDrive/CV_Project', name='CV_Project_evaluation'  )

#performing evaluation on the test split and saving its results on Google Drive
```

After the custom training is complete, the notebook calls our custom model stored in the file best.pt saves in Google Drive. Then, it runs the final evaluation on the test split, never used on the pipeline before. Storing the evaluation results in Google Drive under a folder called “CV_Project_ evaluation”.

Training and evaluation result, metrics and explanations are discussed in: Training and Evaluation Results Discussion.pdf